

Énoncé de l'exercice

Écrire un script Python qui permet de :

1. Donner le principe formel de la programmation dynamique.
2. Aligner les deux séquences d'ADN « GCGTATGC » et « GCTATAC » en appliquant la programmation dynamique.
3. Déterminer l'alignement optimal.
4. Calculer le nombre d'opérations nécessaires.
5. Donner la valeur de la distance (coût) entre les deux séquences.

Question 1:

1. Formal Principle of Dynamic Programming

To find the edit distance between two sequences, we use dynamic programming to fill a table.

Each cell $D(i, j)$ gives the minimum cost to change the first i letters of the first sequence into the first j letters of the second sequence.

We use three possible operations:

- **Insert** a letter (cost = 1)
- **Delete** a letter (cost = 1)
- **Replace** a letter with another (cost = 1)

We fill the table using this rule:

$$D(i, j) = \min \begin{cases} D(i-1, j) + 1 & \text{(delete)} \\ D(i, j-1) + 1 & \text{(insert)} \\ D(i-1, j-1) + 0 \text{ or } 1 & \text{(replace if letters differ)} \end{cases}$$

	$j-1$	j	$j+1$	...
$i-1$	$D(i-1, j-1)$	$D(i-1, j)$	$D(i-1, j+1)$	...
i	$D(i, j-1)$	$D(i, j)$	$D(i, j+1)$	...
$i+1$	$D(i+1, j-1)$	$D(i+1, j)$	$D(i+1, j+1)$	...
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$

Question 2

1 — Define the sequences and the costs

Here we initialize the two DNA sequence inside seq1 and seq2, and we added the cost of each operation:

```
1 # Définir les deux séquences d'ADN
2 seq1 = "GCGTATGC"
3 seq2 = "GCTATAC"
4
5 # Définir les coûts
6 match_cost = 0           # Coût pour un match (lettres identiques)
7 substitution_cost = 2    # Coût pour une substitution (lettres différentes)
8 insertion_cost = 1        # Coût pour une insertion (gap '-')
9 suppression_cost = 1      # Coût pour une suppression (gap '-')
```

2 — Initialization of the alignment matrix

Here we filled Matrix with size(n,m) with zero then we filled the first and the last column from 1..n,1..m:

```
1 # Longueurs des séquences
2 m = len(seq1)
3 n = len(seq2)
4
5 # Initialiser une matrice (m+1) x (n+1) remplie de zéros
6 matrix = np.zeros((n,m))
7
8 # Remplir la première colonne (alignement avec séquence 2 vide)
9 for i in range(m+1):
10     matrix[i][0] = i * insertion_cost
11
12 # Remplir la première ligne (alignement avec séquence 1 vide)
13 for j in range(n+1):
14     matrix[0][j] = j * suppression_cost
```

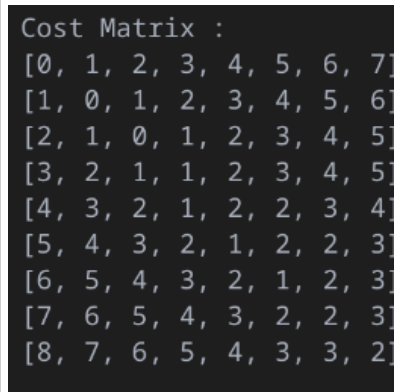
3 — Filling the matrix using dynamic programming

filling the matrix using the **edit distance**

```
1 # Remplir la matrice de programmation dynamique
2 for i in range(1, m+1):
3     for j in range(1, n+1):
4         if seq1[i-1] == seq2[j-1]:
5             cost = match_cost
6         else:
7             cost = substitution_cost
8
9         matrix[i][j] = min(
10             matrix[i-1][j-1] + cost, # diagonale : match ou substitution
11             matrix[i-1][j] + insertion_cost, # haut : insertion
12             matrix[i][j-1] + suppression_cost # gauche : suppression
13         )
```

4 — Displaying the cost matrix

```
1 # Afficher la matrice finale des coûts
2 print("\nMatrice des coûts :")
3 for row in matrix:
4     print(row)
```



```
Cost Matrix :
[0, 1, 2, 3, 4, 5, 6, 7]
[1, 0, 1, 2, 3, 4, 5, 6]
[2, 1, 0, 1, 2, 3, 4, 5]
[3, 2, 1, 1, 2, 3, 4, 5]
[4, 3, 2, 1, 2, 2, 3, 4]
[5, 4, 3, 2, 1, 2, 2, 3]
[6, 5, 4, 3, 2, 1, 2, 3]
[7, 6, 5, 4, 3, 2, 2, 3]
[8, 7, 6, 5, 4, 3, 3, 2]
```

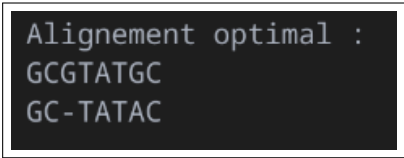
Figure 1: Cost Matrix

Question 3

1 — Retrieving the optimal alignment

Here we use the matrix to check the optimal alignment:

```
1 # Initialiser les chaînes alignées
2 align1 = ""
3 align2 = ""
4
5 # Partir de la fin de la matrice
6 i, j = m, n
7
8 while i > 0 and j > 0:
9     if seq1[i-1] == seq2[j-1] or matrix[i][j] == matrix[i-1][j-1] + substitution_cost:
10         # Match ou substitution
11         align1 = seq1[i-1] + align1
12         align2 = seq2[j-1] + align2
13         i -= 1
14         j -= 1
15     elif matrix[i][j] == matrix[i-1][j] + insertion_cost:
16         # Insertion (gap dans seq2)
17         align1 = seq1[i-1] + align1
18         align2 = '-' + align2
19         i -= 1
20     else:
21         # Suppression (gap dans seq1)
22         align1 = '-' + align1
23         align2 = seq2[j-1] + align2
24         j -= 1
25
26 # Compléter avec des gaps s'il reste des lettres non alignées
27 while i > 0:
28     align1 = seq1[i-1] + align1
29     align2 = '-' + align2
30     i -= 1
31
32 while j > 0:
33     align1 = '-' + align1
34     align2 = seq2[j-1] + align2
35     j -= 1
36
37 # Afficher l'alignement optimal
38 print("\nAlignement optimal :")
39 print(align1)
40 print(align2)
```



```
Alignement optimal :
GCGTATGC
GC-TATAC
```

Figure 2: Optimal Alignment

Question 4,5

1 — Final display of the results

```
1 # Afficher la distance minimale (coût final)
2 distance = matrix[m][n]
3 print(f"\nDistance minimale (coût) entre les deux séquences : {distance}")
```

```
Distance minimale (coût) entre les deux séquences : 3
```

Figure 3: Cost

```
4
5 # Afficher le nombre d'opérations nécessaires
6 operations = len(aligned)
7 print(f"\nNombre d'opérations nécessaires : {operations}")
```

In this example, we need two operations to align the sequences:

- **Deletion:** Removing a 'G' from the first sequence, which costs 1.
- **Substitution:** Replacing a 'G' with an 'A', which costs 2.

Therefore, the total cost of these operations is:

$$1(\text{deletion}) + 2(\text{substitution}) = 3$$